K SHREEKRISHNA UPADHYAYA

Manglore, India | P: +91 9686108613 | upadhyayashreekrishna@gmail.com | linkedin.com/in/kshku | github.com/kshku

PROFILE

Systems programmer focused on low-level C — memory allocators, threading primitives, virtual memory abstraction, hand-written x86_64 assembly, and language runtimes — with a track record of extracting reusable, cross-platform infrastructure from project-specific code and maintaining it as independently-versioned libraries.

EDUCATION

VIVEKANANDA COLLEGE OF ENGINEERING AND TECHNOLOGY, PUTTUR DK, Karnataka
Bachelor of Engineering in Artificial Intelligence & Machine Learning September 2023 – Present
Cumulative GPA (till 6th semester): 8.69/10.0
Relevant Coursework: Operating Systems, Data Structures, Computer Networks

PROJECTS

Sn Project | C, x86_64 Assembly December 2025 – Present

Collection of 13 cross-platform C libraries providing reusable building blocks for native software development, extracted from an experimental game engine and subsequent projects as the repeated-need pattern emerged.

- Designed 7 memory allocator strategies (linear, stack, pool, freelist, queue, ring buffer, frame) behind a unified `SnMemoryAllocator` vtable interface; backed allocators with a two-phase virtual memory subsystem using `mmap/VirtualAlloc` reserve-commit pattern
- Built cross-platform threading primitives (mutex, rwlock, semaphore, spinlock, condvar, thread) using opaque ABI-stable byte buffers that never leak platform types to headers; hand-wrote atomics in x86_64 inline assembly with explicit `lfence/sfence/mfence` placement for memory ordering semantics
- Designed a multi-threaded event tracer with per-thread ring buffers to minimize lock contention, a two-phase begin/commit event API, and Chrome Trace Event Format (JSON) output for visualization in standard trace viewers
- Architected a two-tier logging system: ring buffer for low-latency fast-path enqueue with heap-backed overflow fallback; thread-safety is optional, left to user-provided lock hooks; pluggable sinks via function-pointer abstraction
- Extracted testing infrastructure from Snuk into SnTest, a reusable unit testing framework maintained as part of the ecosystem

Snuk | C April 2026 – June 2026

Embeddable scripting language with expression-oriented semantics and a custom lightweight runtime (6K lines of C).

- Implemented reference-counted memory management with explicit strong/weak pointer separation to eliminate runtime memory leaks
- Built a lexical analyzer, recursive-descent parser, and tree-walking interpreter with first-class functions and closures
- Maintained cross-platform build and test infrastructure via GitHub Actions CI/CD

hash shell | C January 2026 – March 2026

Open-source contributor with 17 merged pull requests to an active codebase.

- Proposed and led a multi-PR refactoring initiative (8+ PRs) extracting helper functions and aligning the codebase with upstream coding standards
- Fixed tab-completion prefix errors and multi-line continuation bugs in quoted strings
- Refactored long functions across the parser, lineedit, prompt, and completion subsystems through PR review

StateFlow | C, raylib September 2025

Visual DFA/NFA simulator with an interactive canvas-based editor for building and testing finite state machines.

- Implemented automaton validity checking: transition completeness per alphabet symbol, single initial state, and accepting-state reachability via depth-first search
- Built a generic dynamic array container using compound-literal macros for type-safe operations without per-type boilerplate
- Rendered each screen to an offscreen texture and blitted it scaled-to-window for resolution-independent UI, with cross-fade transitions between screens

Regex Engine | C December 2025

Regular expression engine supporting grouping, alternation, concatenation, and quantifiers.

- Implemented Thompson-style NFA construction and simulation for pattern matching without backtracking

Additional Projects:
DADS (stuttering dysfluency detection using Wav2Vec2 and a CNN-transformer hybrid model),
FinDiary (personal finance tracker built via Specification-Driven Development),
LC3VM (LC-3 virtual machine), Kilo (terminal text editor).
Winner of Nexathon 2025 (SDIT) and DevHack 2025 (Sahyadri College).

SKILLS

Systems: POSIX APIs, Memory Management, Virtual Memory, Threading & Synchronization, Atomics & Memory Ordering, File Systems, Networking, Dynamic Loading
Languages & Build: C, C++, x86_64 Assembly (inline/MASM), CMake, Python, Go, Java, POSIX Shell
Development: Git, Linux, Typst, Vim, CI/CD (GitHub Actions), gRPC, REST APIs, PostgreSQL